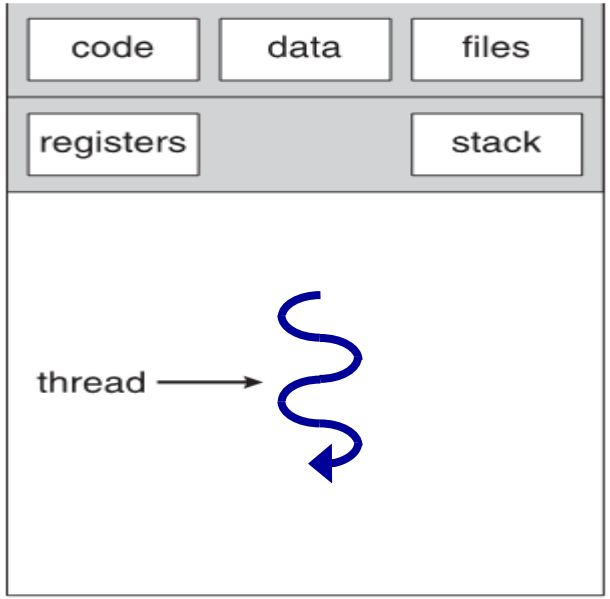
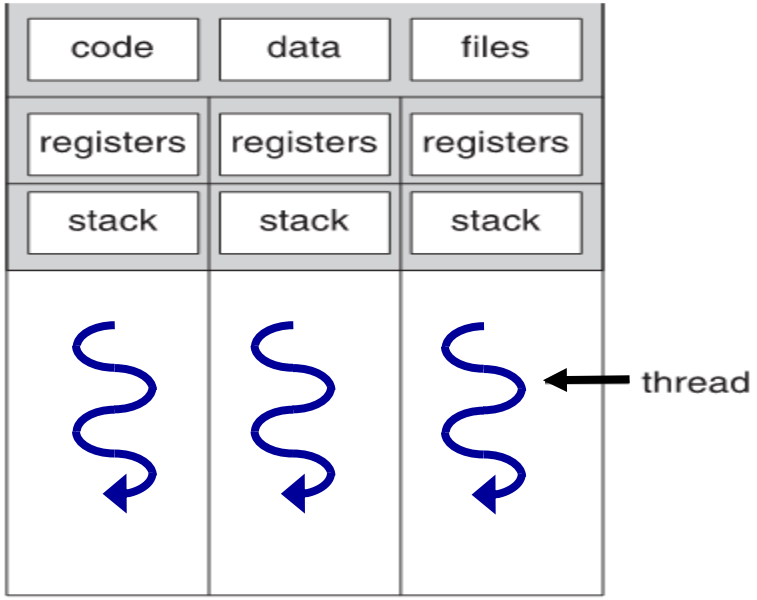
The thread concept

Single and Multi-threaded Processes

- Each Process resides in **its own** address space, **separated** and protected from other processes
- Threads in a Process **share** the process address space - sharing the **Code, Data and Files**
- It is easy to communicate and cooperate but there is no protection from other threads in same process
- Each thread has its own register set and stack and all the necessary data to be able to suspend and freeze it while another thread gets the CPU attention



single-threaded process



multithreaded process

A single threaded process

```
main() {
   …
   f1();
   …
   f2();
   …
   f3();
   …
}

f1() {  …  }
f2() {  …  }
f3() {  … }
```

Once the process, P, is created and the OS allocates it the CPU, the main function is executed and than f1() is invoked – the control (the PC) passes from main to f1

- When f1() returns, the control passes from f1 again to the main
- And then, when f2 is invoked, the control passes from main to f2
- And again to main
- And to f3

- At each time: the control is either in the main, or f1, or f2, or f3
- Not concurrently !!

A multi threaded process

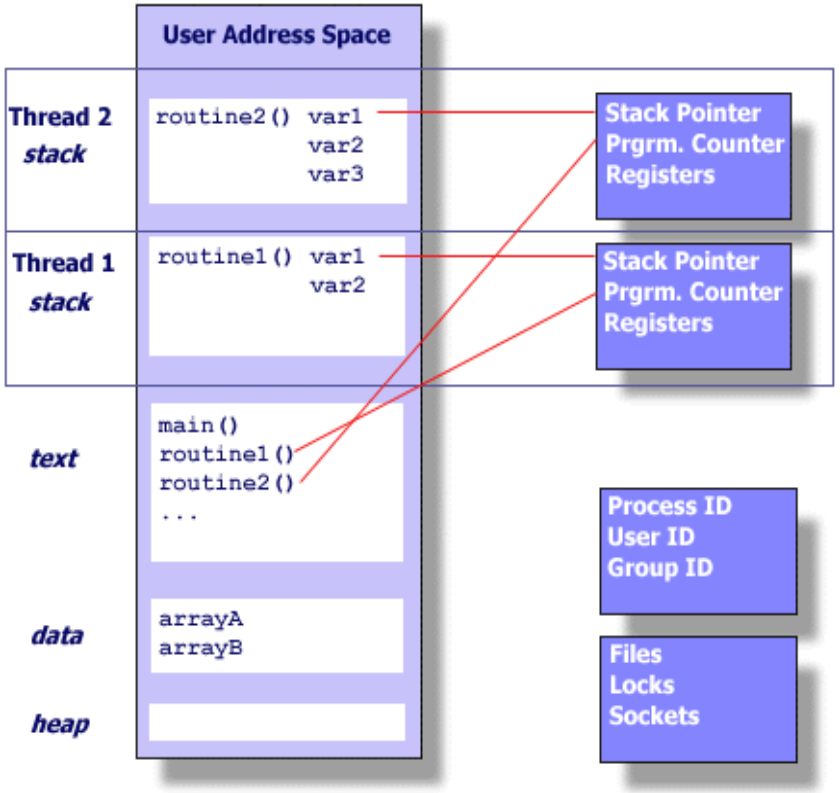
```
main(){
  create_thread(f1 ….) ;
  /* create a new thread (in addition to the main thread),
     and associate it with the code of f1 function) */
  create_thread(f2 …..) ;
  …
  f3();
  …
}

f1() {  …  }
f2() {  …  }
f3() {  … }
```

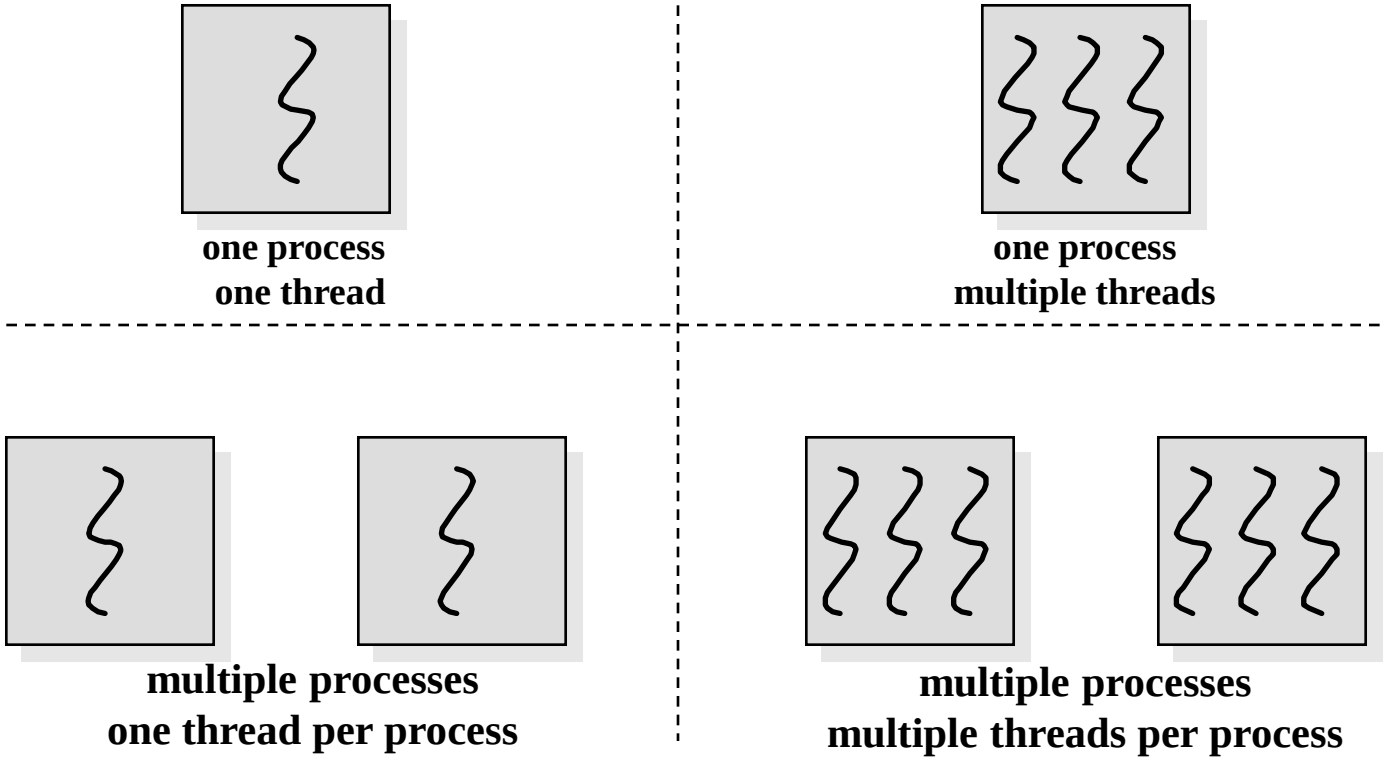
once the process is created and the OS allocates it the CPU, the main function is executed and than **create_thread()** is invoked, and a new thread is created and is associated with the code of f1 function.

- Now, both the main thread and f1 thread can execute concurrently !!
- And when create_thread() is invoked in the 2nd time, yet another thread is created and is associated with the code of f2 function.
- Now, there are 3 threads: the main thread, the threaded associated with f1, and the one associated with the f2 function
- And these 3 threads can execute concurrently !
  - ‘can’ – depends on the availability of a processor (a core) and the scheduling (= allocation of cores) of the OS

Multiple threads in an address space



Threads and Processes



Single Threaded and Multithreaded Process Models